Daymark: a reactive weather and packing planner

CSU44000 Internet Applications – Assignment 1 Liliana Berenguer Jin

1. Requirements

The application takes a destination city and returns a three-day forecast, packing advice (umbrella, and whether to dress for cold, mild or hot weather) and an air-quality warning if any pollutant is above OpenWeather's "Good" band.

Before writing code I mapped each requirement to the OpenWeather endpoint that could satisfy it (Table 1). This settled two things early. Geocoding has to run first, because the forecast and air-pollution endpoints take coordinates rather than a city name. And three of the four requirements come out of the same forecast response, so one request can feed most of the page.

Table 1 – Requirements mapped to OpenWeather APIs

Requirement	OpenWeather API	What it returns
City name → location	Geocoding (`/geo/1.0/direct`)	Latitude and longitude
Three-day forecast (temperature, wind, rain)	5-day / 3-hour Forecast (`/data/2.5/forecast`)	~40 readings at 3-hour intervals
Umbrella and Cold / Mild / Hot advice	Forecast (same response)	Rain and temperature per reading
Pollutant warnings with health risk	Air Pollution (`/data/2.5/air_pollution`)	AQI (1–5) and concentrations of SO_2, NO_2, PM_{10}, $PM_{2.5}$, O_3, CO

The brief also fixes the shape of the system: a Vue client in the browser, a Node server exposing at least one API primitive, and the third-party calls made from the server. That last point is not optional. `fetch` in the browser is subject to the same-origin policy, so a page served from `localhost:3000` cannot call `api.openweathermap.org` directly, and the API key would be visible in the developer tools anyway.

2. System architecture

Figure 1 – System architecture. *Three boxes. Browser (Vue app: reactive state → template). Express server, split into static files from `/public` and the route `GET /api/weather`. OpenWeather with its three endpoints. Double arrow browser ⇄ server labelled "same origin, JSON"; numbered arrows server → OpenWeather for Geocoding, Forecast, Air Pollution; the `.env` API key drawn inside the server box.*

The server does two jobs. `express.static` serves `index.html`, `app.js` and the stylesheet from `/public`, and a single `app.get("/api/weather")` route reads `req.query.city`, calls OpenWeather, and replies with `res.json`. Because the page and the API share an origin there is no CORS configuration at all.

The endpoint is RESTful in the sense that matters: the URI identifies a resource (the packing summary for a city), it is read with `GET`, so repeating the request is safe and idempotent, and the server is stateless. There are no sessions and no database; every

request carries the city it needs and nothing else. I chose that for simplicity, but it is also what makes the API trivial to test with a URL in the browser and what would let it scale behind a load balancer without any sticky-session problems.

The client is thin on purpose. The server returns an already-digested object (`today`, `days`, `packing`, `airQuality`) and the browser only renders it. Putting the rules on the server means they are in one place, can be changed without touching the UI, and are not exposed to the client.

The stack is Node 22, Express 4 and Vue 3 from a CDN with no build step. Node 22 has `fetch` built in and reads `.env` with `--env-file`, so Express is the only dependency.

3. Server design

I built the server in two iterations, which I thought would be useful for separating the question of how to talk to OpenWeather from the question of what to do with its data. Some relevant points from the first iteration:

- It does the minimum: read the city from the query string, build the Geocoding URL with `URLSearchParams`, `await` the reply, take the first match, build the Forecast URL from its `lat` and `lon`, `await` again, and return the readings as `{ time, temperature, description }`. About eighty lines, no validation, no error handling.
- The goal was to get the asynchronous flow right before adding logic. `fetch` runs outside the JavaScript engine as a Web API; `await` suspends the handler and hands control back to the event loop, and the handler resumes from the queue when the response arrives. Node is single-threaded, but the server is never blocked waiting for OpenWeather and can keep serving other requests.
- Writing the two dependent calls with `async/await` rather than `.then()` chains keeps them readable top to bottom, which mattered once a third call was added.
- Returning the raw readings exposed a problem with the phrase "next three days". OpenWeather's timestamps are UTC and the first day is almost always partial, so grouping by UTC date gives the wrong first day for a city like Tokyo. I decided to shift each reading by the `city.timezone` offset before taking its date, show the current partial day separately as "today", and fill the table with the next three full local days.

Figure 2 – Sequence diagram for one search. *Lifelines: User, Vue client, Express, OpenWeather. Submit → client sets `loading`, calls `fetch('/api/weather?city=…')` → server validates → Geocoding request/response → Forecast request/response → Air Pollution request/response inside an `alt` frame (success / fallback) → `res.json` → client sets `result`, template re-renders.*

The second iteration keeps the same chain and adds:

- **Daily summaries.** `makeDailySummaries` groups readings by local date and computes min, max and average temperature, average wind, total rainfall from `rain.3h`, and a `rainExpected` flag set if any reading reports rain or a Rain / Drizzle / Thunderstorm condition.
- **Packing rules.** The Cold / Mild / Hot thresholds from the brief are applied to each day's average. If the days fall into different categories the advice is "pack layers". Umbrella is set if any day expects rain.

- **Air quality.** Each pollutant is compared with the top of OpenWeather's "Good" band and anything above is reported with its band and a short health risk.
- **Error handling.** A `getOpenWeatherJson` wrapper turns network failures and non-2xx replies into errors carrying a status, and the route maps them to the right class of HTTP code (Table 2). The air-pollution call sits in its own `try/catch` so that if that API fails, the forecast is still returned with a note.

Table 2 – Responses from `GET /api/weather`

Status	When
200	Success: `city`, `today`, `days[3]`, `packing`, `airQuality`
400	City missing or over 100 characters
404	Geocoding finds no match
401 / 429	Passed through from OpenWeather (bad key / rate limit)
502	OpenWeather unreachable or returned no readings

Using distinct status codes meant the client needs one `if (!response.ok)` check to handle every failure.

4. Client design

The client is a single Vue instance created with `createApp` and the Options API. `data()` holds four reactive variables, `city`, `loading`, `result` and `error`, and the whole interface is derived from them. I never manipulate the DOM; the template declares what each state looks like and Vue's virtual DOM works out what to re-render when a variable changes. Compared with the `XMLHttpRequest` style of finding an element and setting its `innerHTML`, this removed a whole class of bugs where the page and the data disagree.

Table 3 – Vue features used

Feature	Used for
`v-model.trim`	Two-way binding of the city input
`@submit.prevent`	Run `searchWeather()` without a page reload
`:disabled`	Block the Search button while `loading`
`v-if` / `v-else-if` / `v-else`	Loading, error, empty and result panels; air-quality states
`v-for` with `:key`	Forecast rows and pollutant warnings
`:class`	Colour badges for Cold / Mild / Hot, umbrella advice, AQI dot
`{{ }}`	All text output
`computed`	`cityLabel` from city, state and country
`methods`	`searchWeather` and formatting helpers

`searchWeather` is an `async` method using the Fetch API inside `try / catch / finally`, so `loading` is always reset. A non-OK response puts the server's error message in a panel with a Retry button.

Visually it is a two-panel layout: today on the left, the forecast table with packing and air-quality cards on the right, an SVG icon sprite mapped from OpenWeather's icon codes, and

colour-coded badges. On narrow screens the table collapses into stacked cards. **Figure 3** *(screenshot of the result view)*.

5. Innovative feature

To be written once implemented: what it does, why it is useful, what changed on the server, what changed on the client, screenshot.

6. Testing and reflection

I tested manually: a known city (Dublin, IE), an ambiguous one (Paris vs Paris, US), a made-up city, a missing API key, the air-pollution URL broken on purpose, and Tokyo to check the local-date grouping. The stateless API also meant I could test the server on its own by typing URLs into the browser.

Known limitations: only the first Geocoding match is used, and the air-quality reading is current rather than forecast. The next step would be deploying the server on AWS so it does not have to run locally.

The main thing I took from this is how much of an Internet application is the server reshaping third-party data into exactly what the client needs, and how much simpler the client becomes once the UI is a function of state rather than a sequence of DOM edits.

Appendix A – Source code · **Appendix B – Academic integrity declaration**